



Project Dossier: LMPC Compliance Engine

(SIH26034)

Automated Verification of Mandatory Declarations on Pre-packaged Commodities under the Legal Metrology (Packaged Commodities) Rules, 2011

📌 Problem Statement & Government Context

- **Hackathon:** Smart India Hackathon (SIH)
- **Problem Statement ID:** SIH26034
- **Title:** *Automated Verification of Mandatory Declarations on Pre-packaged Commodities under the Legal Metrology (Packaged Commodities) Rules, 2011*
- **Issuing Ministry:** Ministry of Consumer Affairs, Food & Public Distribution (Department of Consumer Affairs), Government of India
- **Core Problem Solved:** Billions of pre-packaged goods are sold across physical retail and rapid e-commerce platforms (Amazon, Blinkit, Zepto, Flipkart, Swiggy Instamart). Manual market inspection by Legal Metrology Officers is slow, subjective, and physically impossible to scale. Common violations—such as missing **Unit Sale Price (USP)**, altered MRPs, deceptive net quantities, obscured country of origin, and expired goods—directly infringe on Indian consumer rights.

1. 🖥️ Programming Languages Used

LANGUAGE	PRIMARY ROLE IN THE PROJECT	KEY FILES & MODULES
Python 3.10+	Backend API, Multi-Agent LangGraph orchestration, Multimodal AI integration, OpenCV vision processing, deterministic rule evaluation, desktop GUI, and CLI terminals.	backend/app.py , orchestrator.py , desktop_app.py , lmpc_officer_terminal.py
TypeScript (TS)	Type-safe frontend client architecture, data contract interfaces, API communications, and offline-queue logic.	frontend/src/api.ts , offlineQueue.ts , App.tsx
JavaScript / JSX (React 19)	Responsive component rendering, reactive state management, camera scanner integration, and interactive chart visualization.	ImageUploader.tsx , LiveScanner.tsx , AnalyticsDashboard.tsx
HTML5 & CSS3	Clean semantic document structure, glassmorphic dark-mode styling, responsive flex/grid layouts, and printable statutory inspection notices.	index.html , globals.css , report_generator.py
SQL	Relational schema definitions, audit trails, foreign-key relationships for violations, and analytical aggregation queries.	models.py , database.py , migrate_to_neon.py
Bash / PowerShell Batch	Automated field launch scripts and container startup automation.	Launch-Officer-Terminal.bat , docker-compose.yml

2. 🛠️ Tech Stack, Frameworks & Libraries ("What We Used")

A. Backend & Multi-Agent Architecture

- **FastAPI (0.115.12) & Uvicorn:** Asynchronous, high-throughput Python REST API framework.
- **LangGraph (1.2.11) & LangChain:** Stateful Directed Acyclic Graph (DAG) multi-agent orchestrator managing data flow, shared memory, and conditional routing.
- **Google Gemini 3.6 Flash (google-gemini 2.22.0):** High-speed multimodal LLM used for zero-shot structured JSON extraction of packaging text, bounding coordinates, and packaging conditions.
- **ScrapeGraphAI (2.2.2) & Playwright (1.62.0):** Headless browser web scraping engine capable of crawling complex dynamic e-commerce platforms (Amazon, Blinkit, Zepto, Flipkart).
- **OpenCV (opencv-python-headless) & Pillow (PIL):** Image preprocessing, aspect ratio verification, and background-to-text contrast ratio calculation.
- **PyZBar & html5-qrcode:** Dual barcode and QR decoding engines (supporting EAN-13, UPC-A, and 2D codes) for instant product identification.
- **WeasyPrint (63.1):** Headless PDF document generator that turns HTML/CSS inspection notices into official government-grade legal violation notices.

B. Database & Storage Layer

- **SQLAlchemy 2.0 (Async ORM):** Modern asynchronous data persistence layer.
- **Dual Database Capability:**
 - **Local / Edge:** SQLite viaaiosqlite (lmpc_audits.db) for lightweight, offline field work.
 - **Cloud / Production:** Neon Serverless PostgreSQL viaasynccpg with branching and scale-to-zero capabilities.

- **IndexedDB:** Browser client storage (`lmpc_inspector_db`) enabling offline scan queueing when field officers are in underground markets or remote godowns.

C. Frontend & UI Cockpit

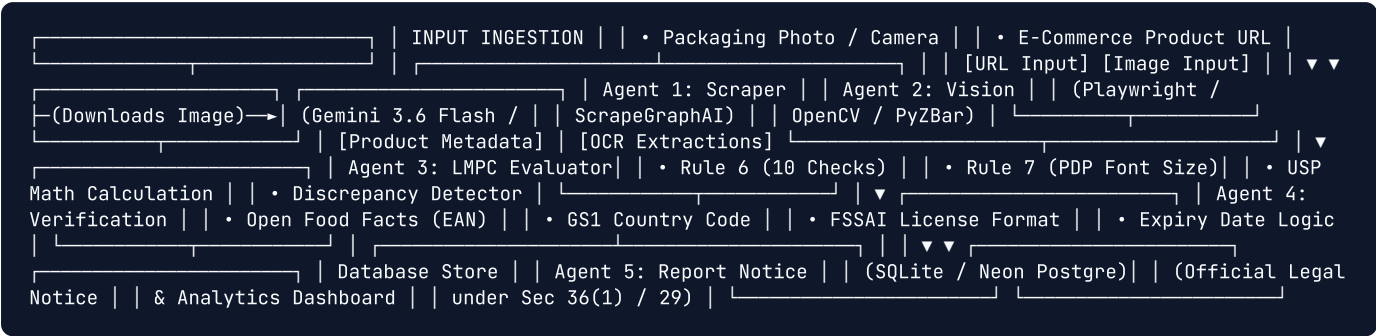
- **React 19 & Vite 8:** Ultra-fast frontend build tooling and component rendering.
- **Framer Motion:** Hardware-accelerated UI animations (laser scanning effects, smooth tabs, collapsible audit cards).
- **Recharts:** Enterprise data visualization for state controllers (brand violation rankings, penalty estimates, compliance trends).
- **Lucide React:** Crisp iconography tailored for official law enforcement dashboards.

D. Alternative Field Cockpits (Officer Tools)

- **Native Desktop Cockpit (`desktop_app.py`):** A Python Tkinter desktop application running offline for inspectors.
- **PowerShell Officer Terminal (`lmpc_officer_terminal.py`):** An ANSI-colored terminal CLI for rapid keyboard-first enforcement auditing without a browser.

3. 🧠 The Architectural Approach Taken

Rather than using a single LLM prompt (which can hallucinate legal rules), this project uses a **hybrid architecture: Multimodal AI Perception + Deterministic Statutory Rule Engine + External Database Verification**.



The 5 Specialized Agents in the Pipeline:

- 🛒 **Agent 1: Scraper Agent (scraper_agent.py)**: Automatically detects the platform (Amazon, Flipkart, Blinkit, Zepto, Swiggy Instamart). Scrapes product title, listed price, claimed MRP, manufacturer claim, origin claim, and high-resolution packaging photos.
- 👁️ **Agent 2: Computer Vision & Multimodal OCR Agent (vision_agent.py)**: Feeds the packaging image to Gemini 3.6 Flash using a strict JSON schema. Extracts all 10 mandatory LMPC fields, label readability status, and bounding confidence scores.
- ⚖️ **Agent 3: LMPC Regulatory Rule Engine (lmpc_evaluator.py)**: **Deterministic Code, Not LLM**: Enforces Indian statutory rules with mathematical precision:
 - 📌 **Rule 6(1)**: Validates all 10 mandatory declarations.
 - 📌 **Rule 6(1)(e) Amendment (2021)**: Computes **Unit Sale Price (USP)** mathematically ($USP = \frac{MRP}{Net\ Quantity}$) and checks whether it is declared in statutory base units (₹/g or ₹/mL for $\leq 1kg/1L$, ₹/kg or ₹/L for $> 1kg/1L$).
 - 📌 **Rule 7 Table 1**: Computes Principal Display Panel (PDP) area and verifies minimum font heights (1.0mm, 1.5mm, 2.0mm, etc.).
 - 📌 **Cross-Modal Discrepancy Detection**: Compares what the e-commerce webpage claims against what the physical packaging photo reveals (e.g., website claims "Made in India", but package reads "Made in China").
- 🔍 **Agent 4: Authenticity & Fraud Verification Agent (verification_agent.py)**: Performs 6 external checks: Barcode Identity (Open Food Facts), GS1 Prefix Check vs Country of Origin, FSSAI 14-digit format, Expiry arithmetic, MRP Anomaly detection, and BIS/ISI Standard checks. Produces a 0-100 Trust Score.
- 📄 **Agent 5: Legal Infraction Notice Generator (report_generator.py)**: Generates an official Government of India Inspection Notice citing statutory sections: Section 18 (Mandatory Declarations), Section 36(1) (Penalties up to ₹25,000 for first offence), Section 29 (Non-standard units), and Section 49 (Director Liability).

4. 🌟 Key Differentiators & Technical Innovations

- Deterministic Legal Precision (Zero Hallucination)**: The LLM is used *strictly for perceptual extraction*. The compliance decision is executed by hardcoded Indian law algorithms.
- Dual-Channel Auditing**: Seamlessly handles physical package images (retail inspection) and live e-commerce product links (digital marketplace monitoring).

3. **Offline Field Inspection Mode:** Uses browser **IndexedDB** queueing. Officers in underground basements or wholesale godowns without Wi-Fi can perform continuous scans, which auto-sync to the central database the moment connectivity returns.
4. **Automated Unit Sale Price (USP) Calculation:** Automatically flags packages that omit USP or express it in deceptive, non-standard units (e.g., using `gms` instead of `g`).
5. **Cross-Modal Discrepancy Detection:** Catches sneaky e-commerce seller tactics where the webpage description does not match the actual physical packaging.

5. 🎯 Top 12 Q&A Defense Sheet (For Interviews / Judges)

Q1 What is the core purpose of this project?

Answer: To automate the enforcement of the *Legal Metrology (Packaged Commodities) Rules, 2011* under the *Legal Metrology Act, 2009*. It eliminates the manual backlog of inspections by using AI vision and automated scraping to audit physical product packages and e-commerce listings for all mandatory statutory declarations.

Q2 What are the 10 mandatory declarations under LMPC Rule 6?

Answer:

1. Name & complete address of Manufacturer / Packer / Importer.
2. Country of Origin.
3. Common or Generic name of the commodity.
4. Net Quantity in legal standard SI units (`g` , `kg` , `ml` , `L` , `m` , `units`).
5. Month and Year of manufacture / packing / import.
6. Best Before or Expiry Date (mandatory for food, cosmetics, perishables).
7. Maximum Retail Price (MRP) inclusive of all taxes.
8. Unit Sale Price (USP) (e.g., ₹/g, ₹/ml, ₹/unit).
9. Complete Consumer Care details (Name, Address, Phone, Email).
10. Dimensions and Principal Display Panel (PDP) compliance.

Q3 Why did you choose a Multi-Agent architecture with LangGraph?

Answer: Regulatory auditing requires separation of concerns. If a single prompt tries to scrape, OCR, check 10 legal rules, calculate mathematical formulas, and check databases, it suffers from context drift and hallucinations. LangGraph allows us to build a stateful, modular pipeline where specialized agents (Scraper → Vision → LMPC Evaluator → Fraud Verifier → Report Generator) do their specific job reliably.

Q4 Why not let an LLM directly decide whether the package is compliant?

Answer: LLMs can hallucinate numerical calculations and legal thresholds. In our architecture, the LLM (Gemini 3.6 Flash) is used *only as a vision perception engine* to extract text into structured JSON. The actual legal evaluation is executed deterministically by Python rule engines enforcing the exact statutory text of the Legal Metrology Rules.

Q5 How do you handle Unit Sale Price (USP) verification?

Answer: The engine uses the formula: `USP = MRP / Net Quantity` . It checks whether the printed USP matches the mathematical calculation within tolerance, and verifies that the correct base unit is declared (`₹/g` or `₹/ml` for packages under 1 kg/1 L; `₹/kg` or `₹/L` for packages over 1 kg/1 L), as mandated by the 2021 LMPC amendment.

Q6 How does the cross-modal e-commerce discrepancy detection work?

Answer: When an e-commerce URL is audited, Agent 1 (Scraper) extracts the seller's textual claims (e.g., "Country of Origin: India", "MRP: ₹199"). Agent 2 extracts the text directly from the product packaging images uploaded by the seller. Agent 3 compares both extractions. If the text says "Made in India" but the packaging label says "Made in China", it immediately flags a **Deceptive E-Commerce Listing** violation under Consumer Protection Rules.

Q7 How does the system detect counterfeit or fraudulent packaging?

Answer: Through Agent 4 (Authenticity Verifier), which runs 6 tests: 1. Verifies the barcode against the global Open Food Facts database; 2. Compares the GS1 barcode country prefix against the declared country of origin; 3. Verifies FSSAI 14-digit license structure and state codes; 4. Performs date arithmetic against the current system time to detect expired commodities; 5. Detects abnormal MRP deviations compared to market averages; 6. Checks BIS/ISI certification requirements.

Q8 How does the solution work for field officers inspecting wholesale markets with no internet?

Answer: The frontend integrates an **IndexedDB-powered Offline Queue**. An inspector scanning products in an underground basement or rural market can continue capturing images and logging records offline. The system queues the inspections locally and automatically synchronizes them to the central government database as soon as network connectivity is restored.

Q9 What database is used and how is it deployed?

Answer: We use an asynchronous SQLAlchemy 2.0 ORM with dual-database flexibility: **Local/Edge:** SQLite (`aiosqlite`) for zero-setup local deployments and offline desktop cockpits; **Production Cloud:** Neon Serverless PostgreSQL (`asyncpg`) with connection pooling and database branching.

Q10 What penalties are cited in the generated Infraction Notice?

Answer: The notices cite statutory sections under the *Legal Metrology Act, 2009*: **Section 36(1)** (Fine up to ₹25,000 for first non-conforming packaging offense; up to ₹50,000 or imprisonment for subsequent offenses); **Section 29** (Fine up to ₹10,000 / imprisonment for quoting non-standard units); and **Section 49** (Corporate liability holding the nominated Company Director legally responsible).

Q11 What web technologies power the user interface?

Answer: React 19, TypeScript, and Vite. The design uses dark glassmorphism, Framer Motion for animations (such as live laser sweeps on package scanning), and Recharts for the state compliance analytics dashboard.

Q12 What alternative interfaces exist besides the web dashboard?

Answer: To ensure adoption across all government office environments, we built: 1. A **Native Desktop Cockpit** (`desktop_app.py`) built with Python Tkinter for standalone Windows workstations; and 2. A **PowerShell Officer Terminal** (`lmpc_officer_terminal.py`) with ANSI color-coding for lightweight, keyboard-driven inspections.